GPU). We will train with `use_ extrinsic=False` in the mini-batch function, so it is learning only from the intrinsic reward. You can plot the individual losses for each of the ICM components and the DQN with the following code. We will log-transform the loss data to keep them on a similar scale.

```
>>> losses_ = np.array(losses)
>>> plt.figure(figsize=(8,6))
>>> plt.plot(np.log(losses_[:,0]),label='Q loss')
>>> plt.plot(np.log(losses_[:,1]),label='Forward loss')
>>> plt.plot(np.log(losses_[:,2]),label='Inverse loss')
>>> plt.legend()
>>> plt.show()
```

As shown in figure 8.17, the DQN loss initially drops and then slowly increases and plateaus. The forward loss seems to slowly decrease but is pretty noisy. The inverse model looks sort of flatlined, but if you were to zoom in, it does seem to very slowly decrease over time. The loss plots look a lot nicer if you set `use_extrinsic=True` and use the extrinsic rewards. But don't feel let down by the loss plots. If we test the trained DQN, you will see that it does a lot better than the loss plots suggest. This is because the ICM and DQN are behaving like an adversarial dynamic system since the forward model is trying to lower its prediction error, but the DQN is trying to maximize the prediction error by steering the agent toward unpredictable states of the environment (figure 8.18).

Figure 8.17. These are the losses for the individual components of the ICM and the DQN. The losses do not smoothly decrease like we're used to with a single supervised neural network because the DQN and ICM are trained adversarially.